

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	P Karthik	Reg. No.:	192524222
Section / Batch:	CSE (AIDS)	Date:	21/07/2026

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I P Karthik (192524222) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:
P Karthik

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO2 AT2

1. A university maintains student records where marks are almost sorted, except for a few recently updated entries. The system needs to sort the data frequently with minimal computation time. The developer is choosing between Selection Sort, Bubble Sort, and Insertion Sort.

a) Analyse all three algorithms in this context.

b) Recommend the most suitable one with justification based on comparisons and swaps.

(a) Analysis of Algorithms

Selection Sort

- Always finds the minimum element in each pass.
- Performs $O(n^2)$ comparisons in all cases.
- Makes fewer swaps but is inefficient for nearly sorted data.

Bubble Sort

- Compares and swaps adjacent elements.
- Best case: $O(n)$ (optimized), Average/Worst: $O(n^2)$.
- Performs many unnecessary swaps.

Insertion Sort

- Inserts each element into its correct position.
- Best case: $O(n)$, Average/Worst: $O(n^2)$.
- Requires very few comparisons and shifts when the data is almost sorted.

(b) Recommendation

- Insertion Sort is the most suitable algorithm.
- The student records are already almost sorted, so only a few entries need repositioning.
- It performs fewer comparisons and shifts than Selection Sort and Bubble Sort.
- It provides the best performance ($O(n)$) for nearly sorted data.
- Therefore, it minimizes computation time and is ideal for frequent updates.

Conclusion

Insertion Sort is the best choice because it is efficient for nearly sorted datasets, performs fewer operations, and is faster than Selection Sort and Bubble Sort in this scenario.

2. A logistics company manages 500 warehouse locations. To optimize delivery routes, the system calculates the distance between every pair of warehouses to identify the closest pair. As the number of warehouses increases, the system becomes slower.

a) Explain how the brute-force approach works in this case.

b) Derive the time complexity.

c) Discuss why scalability is an issue.

(a) Brute Force Approach

- Calculate the distance between every pair of warehouses.
- Compare each distance with the current minimum distance.
- Update the minimum whenever a smaller distance is found.
- After checking all pairs, the closest pair of warehouses is identified.

(b) Time Complexity

- For n warehouses, the number of pairs is $n(n-1)/2$.
- Therefore, the time complexity is $O(n^2)$.
- For 500 warehouses, the algorithm checks $500 \times 499 / 2 = 124,750$ pairs.

(c) Why Scalability is an Issue

- The number of comparisons increases rapidly as the number of warehouses grows.
- More warehouses result in much longer execution time.
- The algorithm becomes inefficient for large datasets.
- Hence, the brute-force approach is suitable only for small datasets, while more efficient algorithms are preferred for large-scale applications.

Conclusion

The brute-force method is simple and guarantees the correct closest pair, but its $O(n^2)$ time complexity makes it unsuitable for large numbers of warehouses.

3. A hospital database stores patient records in an unsorted list because records are frequently added and updated. Doctors need to quickly search for patient details during emergencies.

a) Explain how sequential search works in this scenario.

b) Analyse best, worst, and average case performance.

c) Justify why sorting is not preferred here.

Sequential Search Analysis

(a) How Sequential Search Works

- The search starts from the first patient record.
- Each record is checked one by one until the required patient is found.
- If the record is found, the search stops.
- If all records are checked and no match is found, the patient record does not exist.

(b) Performance Analysis

- Best Case: $O(1)$ – The patient record is found in the first position.
- Average Case: $O(n)$ – The record is found after checking about half of the records.
- Worst Case: $O(n)$ – The record is in the last position or is not present.

(c) Why Sorting is Not Preferred

- Patient records are frequently added and updated.
- Sorting after every update would take extra time and computation.
- Maintaining a sorted list is inefficient in a dynamic database.
- Sequential Search works directly on unsorted data, making it more suitable for this scenario.

Conclusion

Sequential Search is the preferred choice because it works efficiently with unsorted and frequently updated patient records, even though its worst-case time complexity is $O(n)$.

4. A GIS application determines the boundary of a region by identifying outermost points using a brute-force convex hull approach.

a) Explain how the brute-force convex hull algorithm works.

b) Analyse its computational complexity.

c) Discuss why it becomes inefficient for large datasets.

(a) How the Brute Force Convex Hull Algorithm Works

- Consider every pair of points as a possible hull edge.
- For each pair, check whether all other points lie on the same side of the line.
- If all points are on one side, the line is part of the Convex Hull.
- Repeat for all pairs until all boundary edges are identified.

(b) Computational Complexity

- All possible pairs of points are examined.
- For each pair, the remaining points are checked.
- Time Complexity: $O(n^3)$.

(c) Why It Is Inefficient for Large Datasets

- The number of comparisons increases rapidly as the number of points increases.
- More points require checking many additional line pairs.
- This results in high execution time and poor performance.
- Therefore, the brute-force method is suitable only for small datasets, while efficient algorithms like Graham Scan ($O(n \log n)$) are preferred for large datasets.

Conclusion

The brute-force convex hull algorithm correctly identifies the outer boundary but is not efficient for large GIS applications because of its $O(n^3)$ time complexity.

5. A plagiarism detection system compares a pattern against a large document using brute-force string matching. The document length is very large, and the pattern appears rarely.

a) Explain the working of brute-force string matching.

b) Analyse its time complexity.

c) Discuss why performance degrades in this scenario.

(a) Working of Brute Force String Matching

- Compare the pattern with the document starting from the first character.
- If all characters match, the pattern is found.
- If a mismatch occurs, shift the pattern by one position and compare again.

- Repeat until the pattern is found or the document ends.

(b) Time Complexity

- Best Case: $O(n)$ – Pattern matches immediately.
- Average Case: $O(n \times m)$.
- Worst Case: $O(n \times m)$, where n is the document length and m is the pattern length.

(c) Why Performance Degrades

- The document is very large, so many character comparisons are required.
- Since the pattern appears rarely, almost every position in the document must be checked.
- Frequent mismatches lead to repeated comparisons, increasing execution time.
- Therefore, the brute-force method becomes inefficient for large documents.

Conclusion

Brute-force string matching is simple and easy to implement, but its $O(n \times m)$ time complexity makes it unsuitable for large plagiarism detection systems. More efficient algorithms like KMP or Boyer–Moore are preferred.